

Documentation

Analysis

Hackathon 42c x ESTIAM 2026 - "V-Secure & Collaborate"

1. Schema de logs (integration Pole 1)

Ce document definit le format attendu par le module IA & Data (Pole 3 - Sujet B) pour analyser l'audience et predire la retention. Il sert de contrat entre le lecteur video (Pole 1) et le dashboard d'analyse (ce projet).

Format : CSV ou JSON Lines, un evenement par ligne

Champ	Type	Obligatoire	Description
session_id	string	Oui	Identifiant unique de la session de visionnage
user_id	string	Oui	Identifiant de l'utilisateur (anonymisable)
video_id	string	Oui	Identifiant de la video regardee
video_duration_s	float	Oui	Duree totale de la video en secondes
event_type	enum	Oui	play, pause, resume, seek, buffering, abandon, complete
video_time_s	float	Oui	Position de lecture (s) au moment de l'evenement
event_time	datetime	Oui	Horodatage ISO 8601 (ex: 2026-07-01T14:32:05Z)
device	string	Non	desktop, mobile, tablet
playback_rate	float	Non	Vitesse de lecture (1.0 = normale)

Semantique des event_type

- play : demarrage initial de la lecture
- pause : mise en pause (signal fort de decrochage si frequent au meme endroit)
- resume : reprise apres pause
- seek : deplacement dans la timeline (avance rapide = ennui probable ; retour arriere = interet/incomprehension)
- buffering : coupure technique (exclu de l'analyse d'ennui, probleme reseau)
- abandon : fin de session sans avoir termine la video
- complete : video regardee jusqu'au bout

Regle de reconstruction d'une session

Une session est complete des qu'on trouve un evenement abandon ou complete. Le taux de retention de la session = video_time_s du dernier evenement / video_duration_s.

Exemple (CSV)

```
session_id,user_id,video_id,video_duration_s,event_type,video_time_s,event_time,device,playback_rate
s_00042,u_128,v_007,300,play,0,2026-06-30T09:12:00Z,desktop,1.0
s_00042,u_128,v_007,300,pause,145,2026-06-30T09:14:35Z,desktop,1.0
s_00042,u_128,v_007,300,resume,145,2026-06-30T09:14:52Z,desktop,1.0
s_00042,u_128,v_007,300,abandon,168,2026-06-30T09:15:20Z,desktop,1.0
```

Comment le Pole 1 doit envoyer les donnees (2 options)

- Fichier batch (le plus simple pour la demo) : exporter les logs en CSV toutes les X minutes et les déposer dans un dossier partage / uploader dans le dashboard via l'onglet Importer des logs.
- API (pour aller plus loin apres le hackathon) : POST /events avec un JSON respectant les memes champs, un evenement par requete ou par batch.

Pour le ha
n'est pas n

2. Methodologie

2.1 Detection des zones d'ennui

Approche statistique et explicable (pas de boîte noire) : les logs sont decoupees en troncons temporels par video, et un score d'ennui combine le taux de pause, le taux d'avance rapide (seek), le taux d'abandon et la chute de la courbe de retention sur chaque troncon. Les troncons au-dessus d'un seuil sont signales comme zones d'ennui.

- 30% densite de pauses
- 20% densite d'avances rapides (seek)
- 25% nombre d'abandons
- 25% chute de la courbe de retention (derivee negative)

2.2 Modele predictif de retention

Un RandomForestRegressor (scikit-learn) predit le taux de retention final d'une session a partir du comportement observe sur la premiere moitie du visionnage uniquement (pour permettre une prediction anticipee en conditions reelles) et des metadonnees de la video (duree, categorie, device, vitesse de lecture).

Le choix d'un Random Forest plutot qu'un modele plus complexe est volontaire : peu d'hyperparametres a regler, robuste au bruit, et donne une feature importance directement interpretable pour la restitution devant le jury - coherent avec le temps disponible en hackathon.

Les metriques completes (MAE, RMSE, R2, feature importance) sont documentees dans model/metrics_report.md et affichees dans la page Prediction du dashboard.

2.3 Limites connues

- Le modele est entraine sur un dataset synthetique genere pour la demo (data/generate_logs.py), en l'absence de vrais logs du lecteur video au moment du hackathon.
- Des que le Pole 1 fournit de vrais logs conformes au schema, il suffit de relancer python model/train_model.py sur les nouvelles donnees - aucun changement de code necessaire.
- La performance du modele ($R^2 \approx 0.14$) est modeste mais bat clairement une baseline naive : c'est coherent, le comportement precoce d'un spectateur est un signal reel mais partiel, pas deterministe.

3. Comment lancer le projet

Lancer le projet en local

```
pip install -r requirements.txt

# 1. (optionnel) regenerer les donnees de demo
python data/generate_logs.py

# 2. (optionnel) reentrainer le modele
python model/train_model.py

# 3. lancer le dashboard
streamlit run app.py
```

Structure du projet

```
video_analytics/
|-- app.py                # page d'accueil (KPIs, vue d'ensemble)
|-- state.py             # gestion partagee des donnees entre pages
|-- requirements.txt
|-- pages/
|   |-- 1_Zones_dennui.py # detection des zones d'ennui par video
|   |-- 2_Prediction.py   # simulateur de prediction de retention
|   |-- 3_Import_logs.py  # upload de logs reels (integration Pole 1)
|-- docs/
|   |-- Documentation.pdf  # ce document
|-- data/
|   |-- SCHEMA_LOGS.md    # contrat d'integration avec le lecteur video
|   |-- generate_logs.py  # generateur de logs synthetiques
|   |-- viewing_logs.csv  # logs de demo
|   |-- videos_metadata.csv # metadonnees des videos de demo
`-- model/
    |-- analysis.py        # detection de zones d'ennui + agregation en sessions
    |-- train_model.py     # entraînement du modele predictif
    |-- retention_model.joblib # modele entraine
    |-- metrics_report.md  # rapport de performance documente
```